```
> # Maple Transcript: Phonon Probabilistic Flux — TauTimp
  Framework
  # Author: Viorel Popescu
  # Date: March 2026
  # Description: Structured Maple session solving the variable-
  coefficient spherical
  #              wave equation and the acoustic transport equation
  for the phonon field.

  # ============================================================
  # SECTION 1: Physical Parameters
  # ============================================================

  restart;
  with(plots):
  with(DEtools):

  # Physical parameters
  R_val := 4.235518529e22;   # cosmological radius [m]
  V_val := 346;          # speed of sound [m/s]

  printf("TauTimp Phonon Framework\n");
  printf("R = %e m\n", R_val);
  printf("V = %g m/s\n", V_val);

  # ============================================================
  # SECTION 2: Variable-Coefficient Wave Equation
  # ============================================================

  # Define the PDE: (R/r)*Laplacian(f) = (1/V^2)*diff(f,t,t)
  # Under spherical symmetry: Laplacian(f) = diff(f,r,r) + (2/r)*
  diff(f,r)

  pde_wave := (R/r)*(diff(f(r,t),r,r) + (2/r)*diff(f(r,t),r))
            = (1/V^2)*diff(f(r,t),t,t);

  printf("\n--- Variable-Coefficient Wave Equation ---\n");
  print(pde_wave);

  # ============================================================
  # SECTION 3: Separation of Variables  f(r,t) = g(r)*u(t)
  # ============================================================
```

```
# Substituting f = g(r)*u(t) and dividing by g(r)*u(t)
# LHS: (R/r)*(g''(r) + 2/r*g'(r)) / g(r) = const = -omega^2
# RHS: u''(t)/(V^2*u(t)) = -omega^2

# --- 3.1 Temporal ODE ---
ode_temporal := diff(u(t),t,t) + omega^2*V^2*u(t) = 0;
printf("\n--- Temporal ODE ---\n");
print(ode_temporal);

sol_temporal := dsolve(ode_temporal, u(t));
printf("Temporal solution:\n");
print(sol_temporal);

# --- 3.2 Radial ODE (after substitution psi = r*g) ---
# Equation for psi(r) = r*g(r):
#   psi''(r) + (omega^2 * r / R) * psi(r) = 0

ode_radial_psi := diff(psi(r),r,r) + (omega^2/R)*r*psi(r) = 0;
printf("\n--- Radial ODE for psi(r) = r*g(r) ---\n");
print(ode_radial_psi);

# ============================================================
# SECTION 4: Airy Equation Reduction
# ============================================================

# Change of variable: xi = -(omega^2/R)^(1/3) * r
# The equation becomes d^2psi/dxi^2 - xi*psi = 0  (Airy equation)

# Define scaling factor
kappa := (omega^2/R)^(1/3);   # [m^-1] wavenumber scale

printf("\n--- Airy Scaling Factor kappa = (omega^2/R)^(1/3) ---
\n");
print(kappa);

# Airy equation in xi variable
ode_airy := diff(Psi(xi),xi,xi) - xi*Psi(xi) = 0;
printf("\n--- Airy Equation ---\n");
print(ode_airy);

# Maple solution using AiryAi and AiryBi
sol_airy := dsolve(ode_airy, Psi(xi));
```

```
printf("Airy equation solution:\n");
print(sol_airy);

# Physical solution: discard AiryBi (diverges as xi -> -inf, i.e.
, r -> +inf)
# g(r) = (1/r) * C1 * AiryAi( -kappa*r )

g_radial := (C1/r)*AiryAi(-kappa*r);
printf("\n--- Physical Radial Solution g(r) ---\n");
print(g_radial);

# ============================================================
# SECTION 5: Eigenvalue Quantization from BC f(R,0) = 0
# ============================================================

# Condition: AiryAi(-kappa_n * R) = 0
# Let a_n = kappa_n * R = zeros of AiryAi(-a_n) = 0
# => omega_n = (a_n/R)^(3/2) * R^(1/2) = a_n^(3/2) / R ...
# More precisely: kappa_n = a_n/R => (omega_n^2/R)^(1/3) = a_n/R
# => omega_n^2 = a_n^3/R^2 => omega_n = a_n^(3/2)/R

printf("\n--- Eigenvalue Quantization ---\n");
printf("Zeros of AiryAi: a_1 ~ 2.338, a_2 ~ 4.088, a_3 ~
5.521\n");

a_zeros := [2.33810741, 4.08794944, 5.52055983, 6.78670809,
7.94413359];

for n from 1 to 5 do
    a_n := a_zeros[n];
    omega_n := a_n^(3/2) / R_val;
    printf("n=%d: a_%d = %.8f,  omega_%d = %.6e rad/s,  nu_%d =
%.6e Hz\n",
           n, n, a_n, n, omega_n, n, omega_n/(2*Pi));
end do;

# ============================================================
# SECTION 6: Full Airy-Mode Solution f(r,t)
# ============================================================

printf("\n--- Full Airy Mode Solution ---\n");
printf("f_n(r,t) = (1/r) * AiryAi(-a_n*r/R) * [A_n*cos(omega_n*V*
```

```
t) + B_n*sin(omega_n*V*t)]\n");

# Symbolic representation
f_mode := (1/r)*AiryAi(-a_n*r/R)*(A_n*cos(omega_n*V*t) + B_n*sin
(omega_n*V*t));
print(f_mode);

# Plot of first three Airy radial modes at t=0
a1 := 2.33810741:
a2 := 4.08794944:
a3 := 5.52055983:

r_max := R_val:
r_vals := [seq(k*R_val/1000, k=1..999)]:

# Symbolic plot (normalized, using r/R as variable)
p_mode1 := plot(AiryAi(-a1*x), x=0..3, color=blue,
                legend="n=1 Airy mode", title="Airy Radial
Modes"):
p_mode2 := plot(AiryAi(-a2*x), x=0..3, color=red,
                legend="n=2 Airy mode"):
p_mode3 := plot(AiryAi(-a3*x), x=0..3, color=green,
                legend="n=3 Airy mode"):
display(p_mode1, p_mode2, p_mode3,
        title="Phonon Airy radial modes Ai(-a_n * r/R)",
        axes=boxed, labels=["r/R","Ai(-a_n*r/R)"]);

# ============================================================
# SECTION 7: Angular Expansion — Full H(r,theta,t)
# ============================================================

printf("\n--- Full Field H(r,theta,t) via Legendre Expansion ---
\n");
printf("H(r,theta,t) = sum_{ell} sum_n  psi_ell^(n)(r)/r *
LegendreP(ell,cos(theta)) * [A*cos + B*sin]\n");

# Legendre polynomials for illustration
for ell from 0 to 3 do
    printf("P_%d(cos(theta)) = %a\n", ell, simplify(LegendreP
(ell, cos(theta))));
end do;
```

```
# ============================================================
# SECTION 8: Acoustic Wave Equation (d'Alembert)
# ============================================================

printf("\n--- Acoustic (d'Alembert) Wave Equation ---\n");

pde_acoustic := (1/V^2)*diff(f(r,t),t,t) - diff(f(r,t),r,r) = 0;
print(pde_acoustic);

# General solution
sol_acoustic := pdsolve(pde_acoustic, f(r,t));
printf("General solution of acoustic PDE:\n");
print(sol_acoustic);
# Expected: f(r,t) = F1(r + V*t) + F2(r - V*t)

# ============================================================
# SECTION 9: Transport Condition
# ============================================================

printf("\n--- Transport Condition: df/dr = (1/V)*df/dt ---\n");

# For right-moving wave f = F(r + V*t):
#   df/dr = F'(r+Vt),   (1/V)*df/dt = (1/V)*(V*F'(r+Vt)) = F'(r+
Vt)
# => transport condition is satisfied. Right-moving wave
selected.

# For f = F(r - V*t):
#   df/dr = F'(r-Vt),   (1/V)*df/dt = (1/V)*(-V)*F'(r-Vt) = -F'
(r-Vt)
# => df/dr = -df/(V*dt): condition NOT satisfied with + sign.

printf("Transport condition selects: f(r,t) = F(r + V*t)  [right-
moving wave]\n");
printf("At t=0: f(r,0) = F(r) = f0(r)\n");
printf("Then:   f(r,t) = f0(r + V*t)\n");

# ============================================================
# SECTION 10: Polynomial Solution at t = 0
# ============================================================

printf("\n--- Polynomial Solutions for f0(r) ---\n");
```

```
# General degree-2 polynomial: f0(r) = alpha*r^2 + beta*r + gamma
# Conditions:
#   (1) f0(R) = 0
#   (2) f0(0) = 0  [phonon field vanishes at origin]

# From (2): gamma = 0
# From (1): alpha*R^2 + beta*R = 0 => beta = -alpha*R
# Set alpha = 1:

f0_quad := r^2 - R*r;
printf("\nQuadratic solution f0(r) = r^2 - R*r = r*(r - R):\n");
print(f0_quad);

# Verification
printf("f0(0) = %a\n", eval(f0_quad, r=0));
printf("f0(R) = %a\n", simplify(eval(f0_quad, r=R)));
printf("f0'(r) = %a\n", diff(f0_quad, r));
printf("f0'(0) = %a  (inward gradient at origin)\n", eval(diff
(f0_quad,r), r=0));
printf("Minimum at r = R/2: f0(R/2) = %a\n", simplify(eval
(f0_quad, r=R/2)));

# Degree-1 polynomial: f0(r) = beta*(r - R)
f0_lin := r - R;
printf("\nLinear solution f0(r) = r - R:\n");
print(f0_lin);
printf("f0(R) = %a\n", eval(f0_lin, r=R));
printf("f0'(r) = 1 (constant gradient)\n");

# ============================================================
# SECTION 11: Full Time-Dependent Polynomial Solutions
# ============================================================

printf("\n--- Full Time-Dependent Solutions via Transport ---\n")
;

# Right-moving wave: f(r,t) = f0(r + V*t)
f_quad_full := (r + V*t)^2 - R*(r + V*t);
f_lin_full  := (r + V*t) - R;

printf("Quadratic: f(r,t) = (r+Vt)^2 - R*(r+Vt)\n");
print(expand(f_quad_full));
```

```
printf("Linear: f(r,t) = (r+Vt) - R\n");
print(f_lin_full);

# Verify wave equation for quadratic solution
lhs_check := (1/V^2)*diff(f_quad_full, t, t) - diff(f_quad_full,
r, r);
printf("\nVerification of d'Alembert equation for quadratic
f:\n");
printf("(1/V^2)*ftt - frr = %a  (should be 0)\n", simplify
(lhs_check));

# Verify transport condition for quadratic solution
transport_check := diff(f_quad_full, r) - (1/V)*diff(f_quad_full,
t);
printf("Transport condition df/dr - (1/V)*df/dt = %a  (should be
0)\n",
       simplify(transport_check));

# ============================================================
# SECTION 12: Probabilistic Phonon Flux Density
# ============================================================

printf("\n--- Probabilistic Phonon Flux Density rho(r,t) = f(r,t)
/(4*Pi*r^2) ---\n");

# At t = 0
rho_quad_t0 := f0_quad / (4*Pi*r^2);
rho_lin_t0  := f0_lin  / (4*Pi*r^2);

rho_quad_simplified := simplify(rho_quad_t0);
rho_lin_simplified  := simplify(rho_lin_t0);

printf("Quadratic flux at t=0: rho = %a\n", rho_quad_simplified);
printf("Linear flux at t=0:    rho = %a\n", rho_lin_simplified);

# Limit as r -> 0 for linear case
printf("\nLinear flux limit r -> 0: rho ~ -R/(4*Pi*r^2)  [Newton
1/r^2 law]\n");

# Numerical evaluation at r = 1 AU = 1.496e11 m
r_AU := 1.496e11;
rho_lin_AU := eval(subs(R=R_val, rho_lin_simplified), r=r_AU);
```

```
rho_quad_AU := eval(subs(R=R_val, rho_quad_simplified), r=r_AU);

printf("\nAt r = 1 AU = %.3e m:\n", r_AU);
printf("  Linear flux:    rho = %.6e m^-1\n", rho_lin_AU);
printf("  Quadratic flux: rho = %.6e m^-1\n", rho_quad_AU);

# Numerical evaluation at r = 0.1*R
r_01R := 0.1*R_val;
rho_lin_01R := eval(subs(R=R_val, rho_lin_simplified), r=r_01R);
printf("At r = 0.1*R:\n");
printf("  Linear flux: rho = %.6e m^-1\n", rho_lin_01R);

# ============================================================
# SECTION 13: Plots of Polynomial Solutions and Flux
# ============================================================

printf("\n--- Generating Plots ---\n");

# Plot of f0(r) / R^2 vs r/R  (dimensionless)
p_quad := plot((x^2 - x), x=0..1, color=blue, thickness=2,
               legend="f0 = r(r-R)/R^2 quadratic"):
p_lin  := plot((x - 1),   x=0..1, color=red,  thickness=2,
               legend="f0 = (r-R)/R linear"):
display(p_quad, p_lin,
        title="Phonon field f0(r) at t=0  (dimensionless units
r/R)",
        labels=["r/R", "f0/R^2 or f0/R"],
        axes=boxed);

# Plot of probabilistic flux density rho * R^2 vs r/R
# Quadratic: rho = (r-R)/(4*Pi*r) => rho*R = (x-1)/(4*Pi*x)
# Linear:    rho = (r-R)/(4*Pi*r^2) => rho*R^2 = (x-1)/(4*Pi*x^2)

p_flux_quad := plot((x-1)/(4*Pi*x),    x=0.01..1, color=blue,
thickness=2,
                    legend="Quadratic flux rho*R"):
p_flux_lin  := plot((x-1)/(4*Pi*x^2),  x=0.01..1, color=red,
thickness=2,
                    legend="Linear flux rho*R^2"):
display(p_flux_quad, p_flux_lin,
        title="Phonon flux density rho(r,0) at t=0",
        labels=["r/R", "rho (dimensionless)"],
```

```
        axes=boxed, view=[0..1, -5..0.5]);

# ============================================================
# SECTION 14: Comparison Table — Airy vs Polynomial
# ============================================================

printf("\n--- Summary Table ---\n");
printf("%-30s %-35s %-30s\n", "Feature", "Airy Solution",
"Polynomial Solution");
printf("%-30s %-35s %-30s\n", string(30,"-"), string(35,"-"),
string(30,"-"));
printf("%-30s %-35s %-30s\n", "Governing PDE",
       "(R/r)*Laplacian(f)=ftt/V^2",
       "ftt/V^2 - frr = 0");
printf("%-30s %-35s %-30s\n", "Time dependence",
       "cos(omega_n*V*t), sin(omega_n*V*t)",
       "f0(r+V*t)");
printf("%-30s %-35s %-30s\n", "Radial profile",
       "AiryAi(-a_n*r/R)/r",
       "r(r-R) or r-R");
printf("%-30s %-35s %-30s\n", "Flux r<<R",
       "decays as Airy oscillation",
       "~-R/(4Pi*r^2) Newton law");
printf("%-30s %-35s %-30s\n", "Eigenvalues",
       "omega_n = a_n^(3/2)/R",
       "continuous (transport wave)");

# ============================================================
# SECTION 15: Numerical Summary
# ============================================================

printf("\n--- Numerical Summary ---\n");
printf("R           = %.6e m\n", R_val);
printf("V           = %.3f m/s\n", V_val);
printf("R/V         = %.6e s (acoustic crossing time)\n",
R_val/V_val);
printf("omega_1     = %.6e rad/s\n", a_zeros[1]^(1.5)/R_val);
printf("nu_1        = %.6e Hz\n", a_zeros[1]^(1.5)/(R_val*2*evalf
(Pi)));
printf("f0_min      = -R^2/4 = %.6e m^2  (at r=R/2)\n", -
(R_val^2)/4);
```

```
# ============================================================
# END OF TRANSCRIPT
# ============================================================

printf("\n=== Phonon TauTimp Maple Transcript Complete ===\n");
```

$$R_val := 4.235518529 \times 10^{22}$$

$$V_val := 346$$

```
TauTimp Phonon Framework
R = 4.235519e+22 m
V = 346 m/s
```

$$pde_wave := \frac{R\left(\frac{\partial^2}{\partial r^2} f(r,t) + \frac{2\left(\frac{\partial}{\partial r} f(r,t)\right)}{r}\right)}{r} = \frac{\frac{\partial^2}{\partial t^2} f(r,t)}{V^2}$$

```
--- Variable-Coefficient Wave Equation ---
```

$$\frac{R\left(\frac{\partial^2}{\partial r^2} f(r,t) + \frac{2\left(\frac{\partial}{\partial r} f(r,t)\right)}{r}\right)}{r} = \frac{\frac{\partial^2}{\partial t^2} f(r,t)}{V^2}$$

$$ode_temporal := \frac{d^2}{dt^2} u(t) + \omega^2 V^2 u(t) = 0$$

```
--- Temporal ODE ---
```

$$\frac{d^2}{dt^2} u(t) + \omega^2 V^2 u(t) = 0$$

$$sol_temporal := u(t) = c_1 \sin(\omega V t) + c_2 \cos(\omega V t)$$

```
Temporal solution:
```

$$u(t) = c_1 \sin(\omega V t) + c_2 \cos(\omega V t)$$

$$ode_radial_psi := \frac{d^2}{dr^2} \psi(r) + \frac{\omega^2 r \psi(r)}{R} = 0$$

```
--- Radial ODE for psi(r) = r*g(r) ---
```

$$\frac{d^2}{dr^2}\,\psi(r) + \frac{\omega^2\, r\,\psi(r)}{R} = 0$$

$$\kappa := \left(\frac{\omega^2}{R}\right)^{1/3}$$

--- Airy Scaling Factor kappa = (omega^2/R)^(1/3) ---

$$\left(\frac{\omega^2}{R}\right)^{1/3}$$

$$ode_airy := \Psi(2,\xi) - \xi\,\Psi(\xi) = 0$$

--- Airy Equation ---

$$\Psi(2,\xi) - \xi\,\Psi(\xi) = 0$$

Error, (in dsolve) expecting an ODE or a set or list of ODEs. Received Psi(2,xi)-xi*Psi(xi) = 0

Airy equation solution:

$$sol_airy$$

$$g_radial := \frac{C1\ \mathrm{AiryAi}\left(-\left(\frac{\omega^2}{R}\right)^{1/3} r\right)}{r}$$

--- Physical Radial Solution g(r) ---

$$\frac{C1\ \mathrm{AiryAi}\left(-\left(\frac{\omega^2}{R}\right)^{1/3} r\right)}{r}$$

--- Eigenvalue Quantization ---

Zeros of AiryAi: a_1 ~ 2.338, a_2 ~ 4.088, a_3 ~ 5.521

$$a_zeros := [2.33810741, 4.08794944, 5.52055983, 6.78670809, 7.94413359]$$

$$a_n := 2.33810741$$

$$omega_n := 8.440926174 \times 10^{-23}$$

n=1: a_1 = 2.33810741, omega_1 = 8.440926e-23 rad/s, nu_1 = 1.343415e-23 Hz

$$a_n := 4.08794944$$

$$omega_n := 1.951424206 \times 10^{-22}$$

```
n=2: a_2 = 4.08794944,  omega_2 = 1.951424e-22 rad/s,  nu_2 =
3.105788e-23 Hz
```

$$a_n := 5.52055983$$

$$omega_n := 3.062443576 \times 10^{-22}$$

```
n=3: a_3 = 5.52055983,  omega_3 = 3.062444e-22 rad/s,  nu_3 =
4.874030e-23 Hz
```

$$a_n := 6.78670809$$

$$omega_n := 4.174285686 \times 10^{-22}$$

```
n=4: a_4 = 6.78670809,  omega_4 = 4.174286e-22 rad/s,  nu_4 =
6.643582e-23 Hz
```

$$a_n := 7.94413359$$

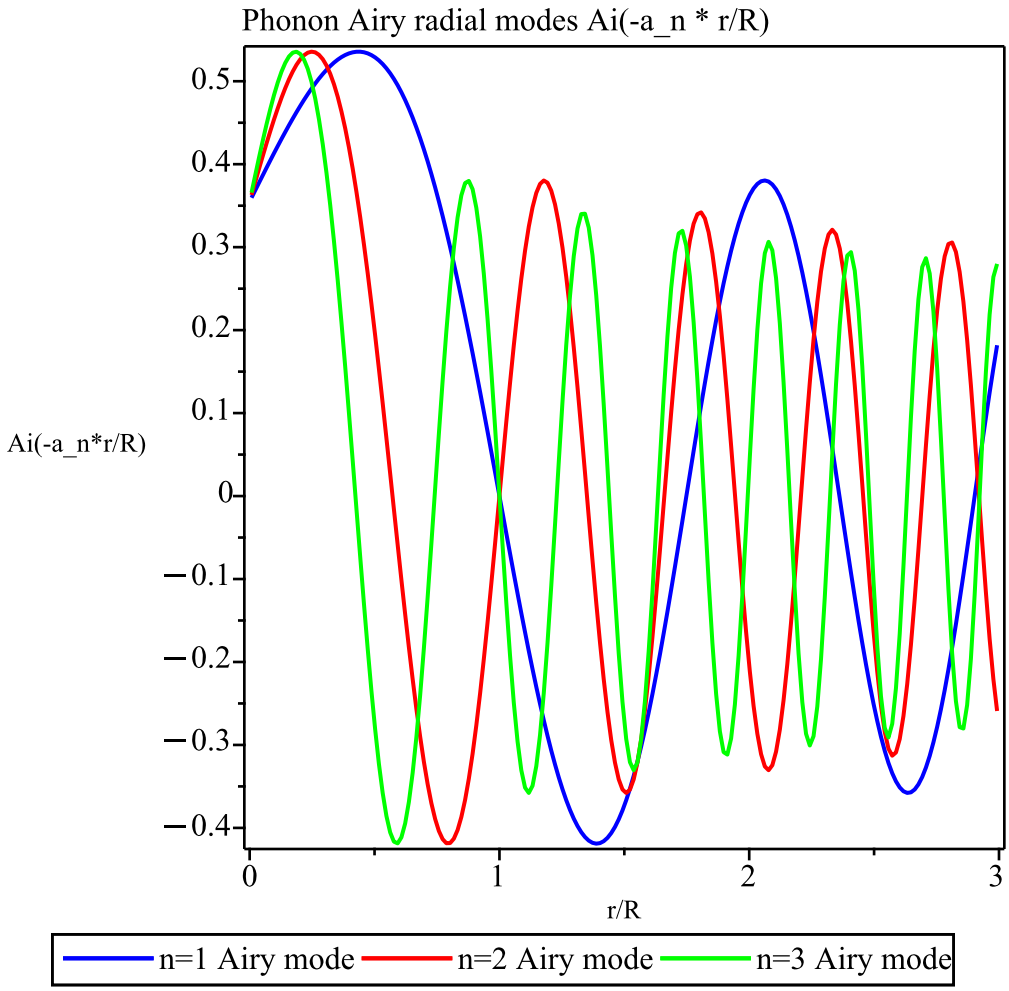
$$omega_n := 5.286438961 \times 10^{-22}$$

```
n=5: a_5 = 7.94413359,  omega_5 = 5.286439e-22 rad/s,  nu_5 =
8.413629e-23 Hz

--- Full Airy Mode Solution ---
f_n(r,t) = (1/r) * AiryAi(-a_n*r/R) * [A_n*cos(omega_n*V*t) + B_n*sin
(omega_n*V*t)]
```

$$f_mode := \frac{1}{r}\left(\text{AiryAi}\left(-\frac{7.94413359\,r}{R}\right)\left(A_n\cos\left(5.286438961\times10^{-22}\,V\,t\right)\right.\right.$$
$$\left.\left.+\,B_n\sin\left(5.286438961\times10^{-22}\,V\,t\right)\right)\right)$$

$$\frac{1}{r}\left(\text{AiryAi}\left(-\frac{7.94413359\,r}{R}\right)\left(A_n\cos\left(5.286438961\times10^{-22}\,V\,t\right)+B_n\sin\left(5.286438961\right.\right.\right.$$
$$\left.\left.\left.\times10^{-22}\,V\,t\right)\right)\right)$$




```
--- Full Field H(r,theta,t) via Legendre Expansion ---
H(r,theta,t) = sum_{ell} sum_n  psi_ell^(n)(r)/r * LegendreP(ell,cos
(theta)) * [A*cos + B*sin]
P_0(cos(theta)) = 1
P_1(cos(theta)) = cos(theta)
P_2(cos(theta)) = 1/4+3/4*cos(2*theta)
P_3(cos(theta)) = 5/2*cos(theta)^3-3/2*cos(theta)

--- Acoustic (d'Alembert) Wave Equation ---
```

$$pde_acoustic := \frac{\frac{\partial^2}{\partial t^2} f(r,t)}{V^2} - \frac{\partial^2}{\partial r^2} f(r,t) = 0$$

$$\frac{\frac{\partial^2}{\partial t^2} f(r,t)}{V^2} - \frac{\partial^2}{\partial r^2} f(r,t) = 0$$

$$sol_acoustic := f(r,t) = f_1(Vt + r) + f_2(Vt - r)$$

General solution of acoustic PDE:

$$f(r,t) = f_1(Vt + r) + f_2(Vt - r)$$

--- Transport Condition: df/dr = (1/V)*df/dt ---
Transport condition selects: f(r,t) = F(r + V*t) [right-moving wave]
At t=0: f(r,0) = F(r) = f0(r)
Then: f(r,t) = f0(r + V*t)

--- Polynomial Solutions for f0(r) ---

$$f0_quad := -Rr + r^2$$

Quadratic solution f0(r) = r^2 - R*r = r*(r - R):

$$-Rr + r^2$$

f0(0) = 0
f0(R) = 0
f0'(r) = -R+2*r
f0'(0) = -R (inward gradient at origin)
Minimum at r = R/2: f0(R/2) = -1/4*R^2

$$f0_lin := r - R$$

Linear solution f0(r) = r - R:

$$r - R$$

f0(R) = 0
f0'(r) = 1 (constant gradient)

--- Full Time-Dependent Solutions via Transport ---

$$f_quad_full := (Vt + r)^2 - R(Vt + r)$$

$$f_lin_full := Vt - R + r$$

Quadratic: f(r,t) = (r+Vt)^2 - R*(r+Vt)

$$V^2 t^2 - RVt + 2Vrt - Rr + r^2$$

Linear: f(r,t) = (r+Vt) - R

$$Vt - R + r$$

$$lhs_check := 0$$

Verification of d'Alembert equation for quadratic f:
(1/V^2)*ftt - frr = 0 (should be 0)

$$transport_check := 2\,V\,t - R + 2\,r - \frac{2\,(V\,t + r)\,V - R\,V}{V}$$

Transport condition df/dr - (1/V)*df/dt = 0 (should be 0)

--- Probabilistic Phonon Flux Density rho(r,t) = f(r,t)/(4*Pi*r^2) ---

$$rho_quad_t0 := \frac{-R\,r + r^2}{4\,\pi\,r^2}$$

$$rho_lin_t0 := \frac{r - R}{4\,\pi\,r^2}$$

$$rho_quad_simplified := \frac{r - R}{4\,r\,\pi}$$

$$rho_lin_simplified := \frac{r - R}{4\,\pi\,r^2}$$

Quadratic flux at t=0: rho = 1/4*(r-R)/r/Pi
Linear flux at t=0:   rho = 1/4*(r-R)/Pi/r^2

Linear flux limit r -> 0: rho ~ -R/(4*Pi*r^2) [Newton 1/r^2 law]

$$r_AU := 1.496 \times 10^{11}$$

$$rho_lin_AU := -0.1506029694$$

$$rho_quad_AU := -2.253020422 \times 10^{10}$$

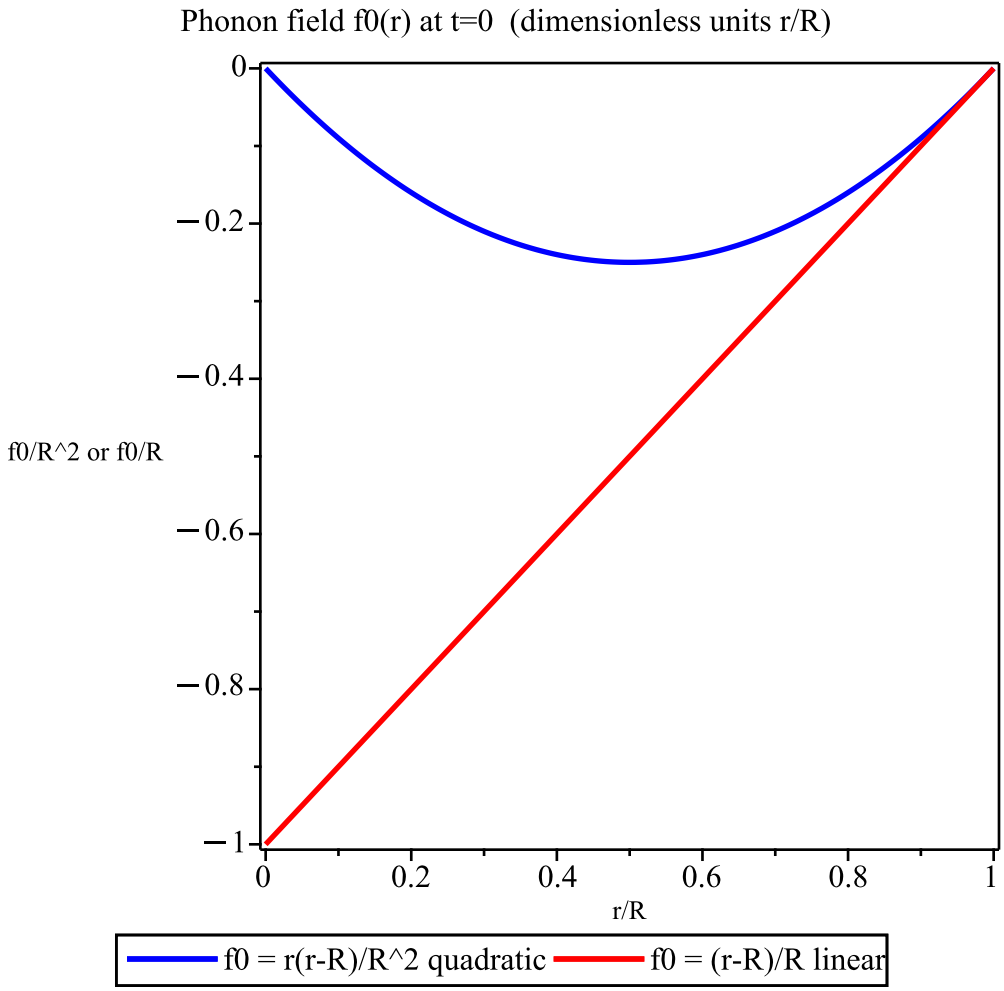
At r = 1 AU = 1.496e+11 m:
  Linear flux: rho = -1.506030e-01 m^-1
  Quadratic flux: rho = -2.253020e+10 m^-1

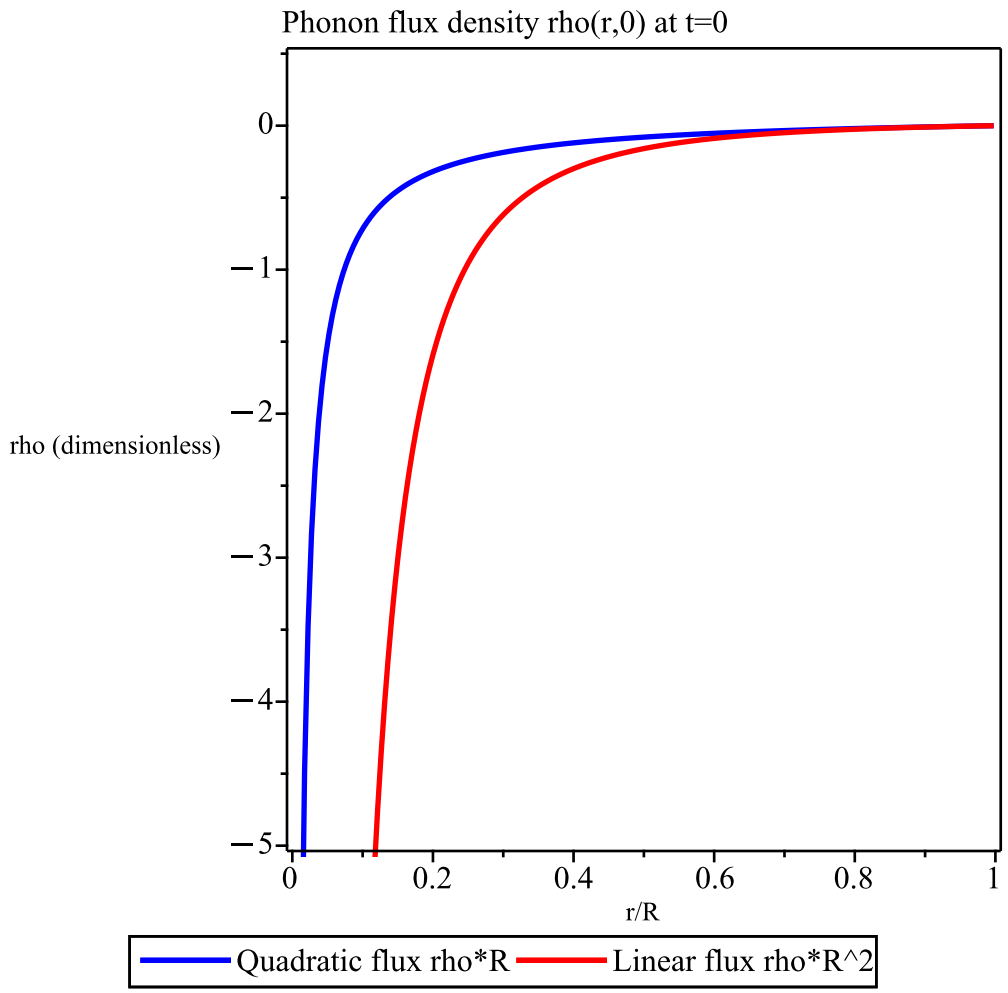
$$r_01R := 4.235518529 \times 10^{21}$$

$$rho_lin_01R := -1.690931675 \times 10^{-22}$$

At r = 0.1*R:
  Linear flux: rho = -1.690932e-22 m^-1

--- Generating Plots ---






```
--- Summary Table ---
Feature                          Airy Solution
Polynomial Solution

Error, (in fprintf) string expected for string format
Governing PDE                    (R/r)*Laplacian(f)=ftt/V^2
ftt/V^2 - frr = 0
Time dependence                  cos(omega_n*V*t), sin(omega_n*V*t)   f0
(r+V*t)
Radial profile                   AiryAi(-a_n*r/R)/r                   r
(r-R) or r-R
Flux r<<R                        decays as Airy oscillation          ~-
R/(4Pi*r^2) Newton law
Eigenvalues                      omega_n = a_n^(3/2)/R
continuous (transport wave)

--- Numerical Summary ---
R           = 4.235519e+22 m
```

```
V            = 346.000 m/s
R/V          = 1.224138e+20 s (acoustic crossing time)
omega_1      = 8.440926e-23 rad/s
nu_1         = 1.343415e-23 Hz
f0_min       = -R^2/4 = -4.484904e+44 m^2  (at r=R/2)

=== Phonon TauTimp Maple Transcript Complete ===
```